
PROBABILISTIC FORECASTING VIA AUTOREGRESSIVE FLOW MATCHING

Anonymous authors
Anonymous submission

ABSTRACT

We introduce FLOWTIME, a generative framework for probabilistic forecasting in multivariate timeseries. Conditioned on historical observations and optional future covariates, FLOWTIME treats forecasting as sampling from the conditional law of future trajectories. The central construction replaces full-horizon transport with an autoregressive factorization of the future joint distribution: a shared flow maps a simple base distribution to each next-step conditional observation distribution under the currently available temporal context. This formulation brings flow matching (FM) into autoregressive probabilistic forecasting and gives a scalable, simulation-free objective for learning the corresponding conditional transports. By combining sequential factorization with transport-based generative modeling, FLOWTIME provides compact parameter sharing, explicit propagation of forecast uncertainty, and extrapolative sample generation while retaining the ability to represent complex conditional predictive distributions. Experiments on stochastic dynamical systems and real-world forecasting benchmarks show that FLOWTIME delivers strong probabilistic forecasts and, in the reported settings, improves on or matches competitive flow-matching and diffusion-based baselines.

1 INTRODUCTION

A central problem in modern machine learning is probabilistic timeseries forecasting: estimating how system trajectories may evolve beyond the observed record while retaining a calibrated description of predictive uncertainty. This distributional formulation is indispensable in scientific, industrial, and societal settings in which decisions depend not only on a point prediction, but also on the range and probability of plausible future outcomes (Lim & Zohren, 2021; Dama & Sinoquet, 2021; Ye et al., 2024).

Deep generative models address this requirement directly by casting forecasting as conditional sampling from a future probability density given past observations (Karl et al., 2016; Rasul et al., 2020; Desai et al., 2021). Within this family, diffusion and score-based generative models (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2020) have substantially improved empirical performance on forecasting benchmarks (Rasul et al., 2021; Tashiro et al., 2021; Kollovich et al., 2023; Meijer & Chen, 2024). Their predictive strength, however, typically comes with considerable computational cost, because training and inference often require many stochastic denoising operations.

Flow Matching (FM) (Liu et al., 2022; Albergo et al., 2023; Lipman et al., 2022) offers a complementary generative paradigm that generalizes diffusion-style constructions while permitting flexible probability paths. Instead of learning an iterative stochastic denoising process over a discretized trajectory, FM learns a deterministic vector field whose continuous flow transports an arbitrary base distribution to the target distribution. With appropriate probability paths, FM avoids handcrafted noise schedules and long ancestral sampling chains, thereby supporting efficient training and generation.

Recent forecasting methods have begun to exploit FM for timeseries modeling (Tamir et al., 2024; Hu et al., 2024; Kollovich et al., 2024), reporting strong empirical performance and improved computational efficiency relative to diffusion-based alternatives. Existing FM forecasters, however, generally learn the conditional distribution of an entire fixed-horizon future window given a fixed-horizon context window. Although this non-autoregressive formulation can generate all future time-points in parallel, it places the statistical burden on a high-dimensional trajectory-level transport, can

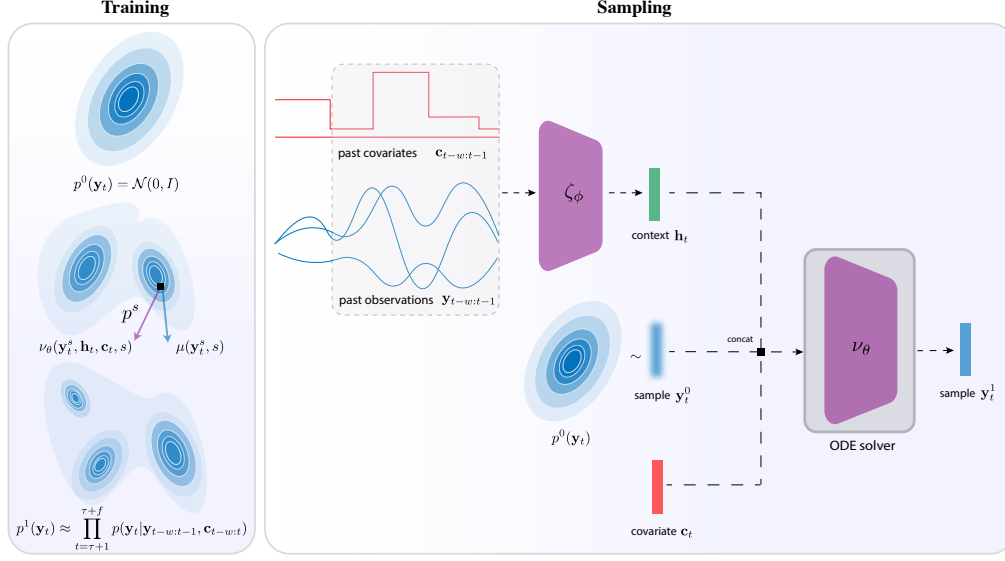


Figure 1: Overview of the FLOWTIME training and sampling procedure. During training, a probabilistic path $(p^s)_{0 \leq s \leq 1}$ transports the base distribution p^0 to the target distribution p^1 through a velocity field μ . The neural velocity field ν_θ regresses the target velocity from a path sample y_t^s at flow step s , a context representation h_t encoding past observations and covariates, and the current covariate c_t . At inference time, sampling begins at p^0 and integrates the learned velocity field with an ODE solver until $s = 1$, producing a draw from the next-step conditional distribution.

extrapolate poorly beyond the training horizon, and may yield less well-calibrated uncertainty. This tension is evident in (Kollovieh et al., 2024), which introduces Gaussian-process-informed priors to simplify and accelerate learning, but consequently restricts the approach to univariate forecasting.

We propose FLOWTIME, an autoregressive FM model for probabilistic forecasting that scales to high-dimensional multivariate datasets without requiring informed priors. The method replaces full-window transport with a sequence of next-step conditional transports: each future observation is sampled from a density determined by a finite history window and the relevant covariates, and all such densities are parameterized by a shared flow from a simple base distribution. This factorization aligns FM with the sequential structure of forecasting, reduces the dimensionality of each transport problem, supports generation over successive horizons, and makes uncertainty propagation explicit through the chain of conditional distributions.

The paper makes three contributions. First, it formulates probabilistic forecasting as autoregressive flow matching, decomposing the future trajectory distribution into tractable next-step conditional transports. Second, it specifies training and sampling procedures that preserve parallel teacher-forced optimization while producing forecasts sequentially at inference time. Third, it evaluates the resulting model on simulated stochastic dynamical systems and real-world datasets, establishing FLOWTIME as a practical and competitive method for probabilistic timeseries forecasting, with especially pronounced accuracy and extrapolation gains in the reported dynamical-system experiments.

2 BACKGROUND: FLOW MATCHING

Flow matching (FM) (Lipman et al., 2022) is a generative-modeling framework that enables scalable training of continuous normalizing flows (CNFs) (Chen et al., 2018; Grathwohl et al., 2018). CNFs parameterize transformations between probability distributions through ordinary differential equations (ODEs), thereby inducing invertible maps that retain tractable likelihood evaluation.

As in CNFs, FM defines a time-dependent diffeomorphic map on the data space Ω , denoted by the flow $\psi: [0, 1] \times \Omega \rightarrow \Omega$. This flow carries a source sample $\mathcal{X}^0 \sim p$ to a terminal sample

$\mathcal{X}^1 := \psi(\mathcal{X}^0, 1)$ distributed according to the target distribution q . The transformation is governed by the initial value problem

$$d\psi(\mathcal{X}, s) = \mu(\psi(\mathcal{X}, s), s)ds, \quad \psi(\mathcal{X}, 0) = \mathcal{X}, \quad s \in [0, 1] \quad (1)$$

where $\mu: [0, 1] \times \Omega \rightarrow \Omega$ is the velocity field that induces a probability path $(p^s)^{0 \leq s \leq 1}$ over Ω with initial marginal $p^0 = p$. The corresponding learning problem is to identify a vector field whose terminal marginal satisfies $p^1 = q$.

Standard CNFs learn the vector field through likelihood maximization and therefore require both solving and differentiating through Eq. (1) during training. FM avoids this training bottleneck by estimating a parameterized vector field ν_θ through regression against vector fields associated with fixed conditional probability paths. This construction yields the tractable conditional flow matching (CFM) objective

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{s \sim \mathcal{U}[0,1], \mathcal{X}^1 \sim q(\mathcal{X}^1), \mathcal{X} \sim p^s(\mathcal{X}|\mathcal{X}^1)} \|\nu_\theta(\mathcal{X}, s) - \mu(\mathcal{X}, s|\mathcal{X}^1)\|^2. \quad (2)$$

After optimization, samples from q are generated by solving Eq. (1) with the learned field ν_θ . Because the objective removes backpropagation through the ODE solver from training, FM supports stable and scalable learning in high-dimensional generative settings, including image and video modeling (Esser et al., 2024; Polyak et al., 2024). Alternative probability-path constructions and conditioning schemes further show that several transport-based generative models can be interpreted within the same formal framework (Liu et al., 2022; Albergo et al., 2023; Tong et al., 2023).

3 AUTOREGRESSIVE FLOW MATCHING

Problem Setting Let $\mathbf{y}_\tau \in \mathbb{R}^n$ denote an n -dimensional observation at time τ . Given the most recent l observations, including the current observation, $\mathbf{Y}_l = \{\mathbf{y}_{\tau-l}, \dots, \mathbf{y}_\tau\}$, the forecasting task is to draw the next f values $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$ from their predictive distribution. The model may additionally condition on covariates $\mathbf{C} = \{\mathbf{c}_{\tau-l}, \dots, \mathbf{c}_{\tau+f}\}$, where $\mathbf{c}_\tau \in \mathbb{R}^c$. Formally, for a dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$, the objective is to learn a conditional sampler for $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$.

FLOWTIME factorizes this conditional distribution autoregressively across future time steps under a Markov assumption of order w :

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \quad (3)$$

where $w \leq l$ is the history size. This decomposition converts full-trajectory density modeling into the repeated estimation of the next-step conditional distributions in Eq. (3). Rather than fitting these conditionals by maximum likelihood, as in many autoregressive forecasting models, FLOWTIME estimates each conditional density through flow matching (Lipman et al., 2022).

This factorization is the principal structural contribution of the method. It substitutes a collection of lower-dimensional conditional transports $p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ for a single high-dimensional trajectory transport, while preserving a joint predictive distribution through their product. The same structure enables teacher-forced training of individual time steps in parallel, exposes a direct capacity-complexity trade-off through the window size w , and makes temporal uncertainty propagation explicit because each sampled prediction becomes part of the conditioning context for later horizons.

3.1 CONSTRUCTING A TARGET PROBABILITY PATH

For each future time step $t \in [\tau+1, \tau+f]$, we define a probability path $(p^s(\mathbf{y}_t))_{0 \leq s \leq 1}$ with source marginal $p^0(\mathbf{y}_t) = \mathcal{N}(0, I)$ and terminal marginal $p^1(\mathbf{y}_t) \approx \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \approx p_{\mathcal{D}}$, where $p_{\mathcal{D}}$ denotes the empirical distribution. A flow field $\psi: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ generates the path along the flow variable s according to the ODE

$$d\psi(\mathbf{y}_t, s) = \mu(\psi(\mathbf{y}_t, s), s)ds, \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t \quad s \in [0, 1] \quad (4)$$

where $\mu: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ is the velocity field selected so that the induced probability path satisfies the continuity equation.

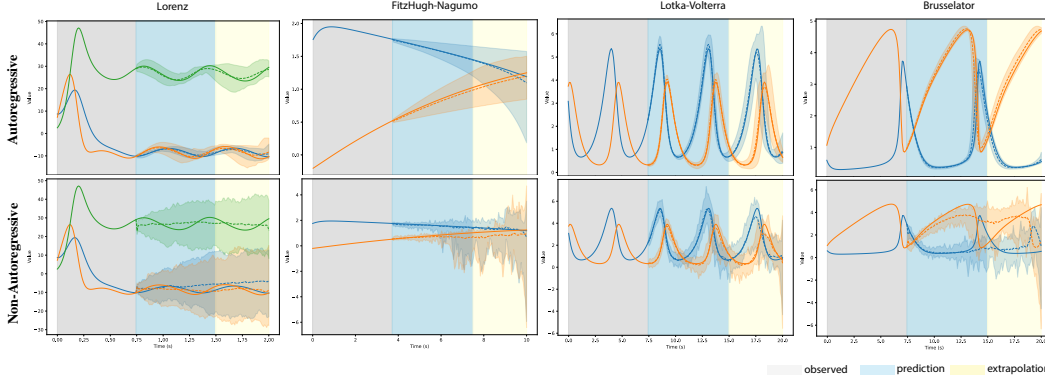


Figure 2: Forecast samples comparing autoregressive flow matching with standard flow matching on four stochastic dynamical systems. Solid curves denote ground truth, and dashed curves denote the predictive mean with the 95% confidence interval. Each panel includes both the prediction and extrapolation regimes, illustrating how the autoregressive factorization shapes trajectory-level uncertainty beyond the observed window.

To obtain a tractable regression objective, we represent the marginal probability path as a mixture of conditional paths:

$$p^s(\mathbf{y}_t) = \int p^s(\mathbf{y}_t|\mathbf{z})p(\mathbf{z})d\mathbf{z} \quad (5)$$

where $\mathbf{z} \sim \pi_{0,1}(\mathbf{z})$ is a conditioning variable drawn from a data coupling $\pi_{0,1}$. Theorem 2 of (Lipman et al., 2022) establishes that regression onto the conditional velocity fields $\mu(\mathbf{y}_t, s|\mathbf{z})$ associated with $p^s(\mathbf{y}_t|\mathbf{z})$ recovers the corresponding marginal velocity field $\mu(\mathbf{y}_t, s)$, which justifies learning through conditional path samples.

Following (Tong et al., 2023), we set $\mathbf{z} = \{\mathbf{y}_t^0, \mathbf{y}_t^1\}$ with $p(\mathbf{z} = (\mathbf{y}_t^0, \mathbf{y}_t^1)) = p^0 \times p^1$, where $\mathbf{y}_t^0$ and $\mathbf{y}_t^1$ are sampled from p^0 and p^1 , respectively. This choice defines the conditional path

$$p^s(\mathbf{y}_t|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I) \quad (6)$$

with endpoint marginals satisfying $p^0(\mathbf{y}_t|\mathbf{z}) = p^0(\mathbf{y}_t)$ and $p^1(\mathbf{y}_t|\mathbf{z}) = p^1(\mathbf{y}_t)$ as $\sigma^2 \rightarrow 0$. The associated conditional velocity field is $\mu(\mathbf{y}_t, s|\mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$. This construction yields straight Euclidean probability paths and closed-form target velocities, which are central to the computational simplicity of the method. After the velocity field has been learned, samples from $p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ are obtained by solving the ODE in Eq. (4) until $s = 1$.

3.2 TRAINING

As in standard FM, we learn the velocity field by fitting a neural approximation ν_θ with parameters θ using the conditional flow matching objective in Eq. 2. To condition the field on the autoregressive history, we introduce a context vector $\mathbf{h}_t \in \mathbb{R}^h$ for time t :

$$\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1}) \quad (7)$$

where $\zeta: \mathbb{R}^{w \times n} \times \mathbb{R}^{w \times c} \rightarrow \mathbb{R}^h$ is a neural encoder with parameters ϕ . The resulting conditional training objective is

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{y}_t^s \sim p^s(\mathbf{y}_t|\mathbf{z})} \|\mu(\mathbf{y}_t^s, s|\mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2 \quad (8)$$

Training draws an observation at time t , its context window, and its covariates from $\mathcal{D}$, and jointly optimizes ϕ and θ by minimizing $\mathcal{L}(\theta, \phi)$ with stochastic gradient descent. The procedure is teacher-forced (Williams & Zipser, 1989): ground-truth past observations, rather than previously generated predictions, define the autoregressive context during training. Consequently, all training time steps can be processed in parallel, and training-time error accumulation is avoided. Algorithm 1 summarizes the resulting autoregressive flow matching training procedure.

Algorithm 1 Training: Autoregressive Flow Matching.

Require: Dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$, window size w , networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
- 2: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$ and $(\mathbf{y}_t^1, \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \sim \mathcal{D}$
- 3: Set $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$
- 4: Sample $s \sim \mathcal{U}(0, 1)$ and $\mathbf{y}_t^s \sim \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I)$
- 5: Compute target: $\mu(\mathbf{y}_t^s, s|\mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$
- 6: Compute loss: $\mathcal{L}_t = \|\mu(\mathbf{y}_t^s, s|\mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2$
- 7: Update θ, ϕ
- 8: **end while**
- 9: **return** ν_θ, ζ_ϕ

Algorithm 2 Sampling: Autoregressive Flow Matching.

Require: Past observations $\mathbf{Y}_l$, covariates $\mathbf{C}$, forecast horizon f , window size w , networks ν_θ, ζ_ϕ

- 1: Initialize $\mathbf{Y}_f \leftarrow \mathbf{Y}_l$
- 2: **for** $t = \tau + 1$ **to** $\tau + f$ **do**
- 3: Set context: $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$, using the last w entries in $\mathbf{Y}_f$
- 4: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$
- 5: Solve $\frac{d\psi(\mathbf{y}_t, s)}{ds} = \nu_\theta(\psi(\mathbf{y}_t, s), \mathbf{h}_t, \mathbf{c}_t, s), \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t^0, \quad s \in [0, 1]$
- 6: Append $\psi(\mathbf{y}_t, 1)$ to $\mathbf{Y}_f$
- 7: **end for**
- 8: **return** $\mathbf{Y}_f$

3.3 INFERENCE

After training, the learned model produces probabilistic forecasts for new observations $\mathbf{Y}_l$ and corresponding covariates $\mathbf{C}$ over the next f timestamps. Inference follows Algorithm 2: at each horizon, the model forms the current rolling context, samples a base point, integrates the learned velocity field to obtain the next observation, and appends that draw to the context used at subsequent horizons. Repeating this autoregressive sampling procedure yields Monte Carlo samples from the predictive distribution, from which empirical quantiles summarize forecast uncertainty.

4 EXPERIMENTS

We evaluate FLOWTIME in two complementary forecasting settings: controlled stochastic dynamical systems, where extrapolation beyond the training window can be examined directly, and established real-world datasets, where performance can be compared with statistical, diffusion-based, and flow-matching baselines. Together, these experiments test whether the autoregressive FM factorization improves predictive accuracy and probabilistic calibration while remaining competitive on standard timeseries benchmarks.

4.1 DYNAMICAL SYSTEMS

Data We evaluated FLOWTIME on forecasting tasks derived from five stochastic dynamical systems: stochastic versions of the Lorenz system (Lorenz, 1963), FitzHugh–Nagumo model (FitzHugh, 1961), Lotka–Volterra system Lotka (1925), Brusselator (Lefever & Prigogine, 1968), and Van der Pol oscillator (van der Pol, 1926). For each system, trajectories were generated by numerically solving the corresponding stochastic differential equation under randomly sampled initial conditions and Brownian motion realizations. Appendix A.1 specifies the systems, parameter choices, and data-generation protocol.

Implementation details In the dynamical-system experiments, ζ_ϕ is a two-layer bi-directional LSTM with 64 hidden units, and ν_θ is an MLP with three hidden layers of size 64. The flow step s is encoded as a 16-dimensional Fourier positional embedding using a fixed set of frequencies, and the context length w is set equal to the prediction length. Models were trained with Adam using a

learning rate of 0.003 and a batch size of 128. Training to convergence required ~ 7 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines To isolate the effect of the autoregressive factorization, we compare FLOWTIME with a non-autoregressive flow-matching baseline that directly models the joint distribution of future observations conditioned on past observations and covariates, i.e. $p^1 = p(\mathbf{Y}_f | \mathbf{Y}_l, \mathbf{C})$. This baseline is a direct ablation of the structural choice introduced by FLOWTIME. Its formulation closely resembles the method proposed in (Kollovich et al., 2024); Appendix B gives the full specification.

Evaluation To evaluate extrapolation beyond the training target window, each trajectory is partitioned into three contiguous segments: (1) an observation window used as historical input; (2) a prediction window available during training as the supervised target; and (3) an extrapolation window withheld entirely during training. We report the continuous rank probability score (CRPS) (Winkler et al., 1996) to assess probabilistic forecast quality and the normalized root mean square error (NRMSE) (Hyndman & Koehler, 2006) to assess predictive accuracy.

Table 1: Test-set performance of the autoregressive factorization (FLOWTIME) and the non-autoregressive (Non-AR) flow-matching baseline on five stochastic dynamical systems. CRPS and NRMSE are reported separately for the prediction window and the extrapolation window; lower values indicate better performance. Each entry gives the mean and standard deviation across five random seeds.

System	Factorization	Prediction		Extrapolation	
		CRPS $\downarrow$	NRMSE $\downarrow$	CRPS $\downarrow$	NRMSE $\downarrow$
Lorenz	Non-AR	0.147 $\pm$ 0.012	0.242 $\pm$ 0.018	0.120 $\pm$ 0.009	0.301 $\pm$ 0.024
	FLOWTIME	0.090 $\pm$ 0.008	0.017 $\pm$ 0.003	0.137 $\pm$ 0.011	0.048 $\pm$ 0.006
FitzHugh-Nagumo	Non-AR	0.165 $\pm$ 0.014	0.131 $\pm$ 0.011	0.214 $\pm$ 0.016	0.306 $\pm$ 0.022
	FLOWTIME	0.144 $\pm$ 0.013	0.090 $\pm$ 0.007	0.146 $\pm$ 0.012	0.278 $\pm$ 0.019
Lotka-Volterra	Non-AR	0.130 $\pm$ 0.011	0.266 $\pm$ 0.019	0.136 $\pm$ 0.012	0.502 $\pm$ 0.033
	FLOWTIME	0.126 $\pm$ 0.010	0.189 $\pm$ 0.015	0.123 $\pm$ 0.009	0.339 $\pm$ 0.026
Brusselator	Non-AR	0.217 $\pm$ 0.016	0.658 $\pm$ 0.042	0.238 $\pm$ 0.019	1.046 $\pm$ 0.057
	FLOWTIME	0.125 $\pm$ 0.011	0.031 $\pm$ 0.004	0.109 $\pm$ 0.008	0.023 $\pm$ 0.003
Van der Pol	Non-AR	0.143 $\pm$ 0.012	0.161 $\pm$ 0.014	0.211 $\pm$ 0.018	0.510 $\pm$ 0.031
	FLOWTIME	0.225 $\pm$ 0.017	0.051 $\pm$ 0.005	0.282 $\pm$ 0.021	0.101 $\pm$ 0.009

Results Table 1 shows that the autoregressive factorization in FLOWTIME provides a consistent accuracy advantage across the stochastic dynamical systems. In the prediction window, FLOWTIME attains lower NRMSE than the non-autoregressive baseline on all five systems, with especially large reductions on Lorenz (0.017 vs 0.242) and Brusselator (0.031 vs 0.658), corresponding to error reductions of 93% and 95%. The same factorization also improves CRPS on four of the five systems; Van der Pol is the exception, where the non-autoregressive baseline attains lower CRPS despite higher NRMSE. The extrapolation results preserve the accuracy pattern: FLOWTIME has lower NRMSE on every system, including a 98% reduction on Brusselator (0.023 vs 1.046). These comparisons substantiate the central empirical claim that autoregressive conditional transports capture dynamical structure more effectively than a single full-window transport in these evaluated systems, particularly when forecasts are rolled beyond the supervised prediction window.

4.2 REAL-WORLD DATASETS

Data We further evaluated FLOWTIME on univariate and multivariate forecasting tasks with hourly and daily sampling frequencies from GluonTS (Alexandrov et al., 2020). The benchmark suite comprises Electricity (Lai et al., 2018), Solar (Lai et al., 2018), Exchange (Dua et al., 2017), Traffic (Dua et al., 2017), and Wikipedia (Gasthaus et al., 2019). Appendix A.2 provides dataset details.

Implementation details For the real-world datasets, ζ_ϕ is a three-layer bi-directional LSTM with 64 hidden units. The velocity network ν_θ is a residual neural network with five residual blocks implemented using 1-D convolutional layers and gated activation functions (Van Den Oord et al., 2016; Kong et al., 2020). This architecture follows the network design in (Rasul et al., 2021) but uses fewer residual blocks. Fourier positional embeddings encode the flow step s into a 32-dimensional vector using a fixed set of frequencies, and the same embedding mechanism encodes time-dependent covariates $\mathbf{c}$. Models were trained with Adam using a learning rate of 0.001 and a batch size of 128. Depending on the dataset, convergence required 40 – 100 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines We compare FLOWTIME with established statistical, neural, diffusion-based, and flow-matching forecasters. The statistical baselines are Seasonal Naive (SN), AutoARIMA, and AutoETS (Hyndman et al., 2008); the neural baselines are DeepAR (Salinas et al., 2020) and WaveNet (Van Den Oord et al., 2016). We also include the diffusion baselines TSDiff (Kollovich et al., 2023), SSSD (Alcaraz & Strodtzoff, 2022), and TimeGrad (Rasul et al., 2021), together with the flow-matching baseline TSFlow (Kollovich et al., 2024).

Table 2: Test-set CRPS comparison on five real-world forecasting datasets; lower values indicate better probabilistic forecast quality. For FLOWTIME, entries report the mean and standard error over five independent training and evaluation runs. Baseline results are taken from (Kollovich et al., 2024). Best scores are shown in **bold**, and second-best scores are underlined.

Method	Electr.	Exchange	Solar	Traffic	Wikipedia
SN	0.060±0.000	0.013±0.000	0.512±0.000	0.221±0.000	0.423±0.000
ARIMA	0.344±0.000	0.013±0.000	0.558±0.003	0.486±0.000	0.654±0.000
ETS	0.056±0.000	0.008±0.000	0.550±0.000	0.492±0.000	0.651±0.000
DeepAR	0.051±0.000	0.013±0.004	0.429±0.015	0.103±0.000	0.215±0.003
WaveNet	0.058±0.008	0.012±0.001	0.360±0.000	0.099±0.000	0.207 ±0.000
CSDI	0.051±0.000	0.013±0.000	0.360±0.000	0.152±0.000	0.318±0.012
SSSD	0.048±0.000	<u>0.010</u> ±0.000	0.354±0.024	0.107±0.002	<u>0.209</u> ±0.000
TSFlow	<u>0.045</u> ±0.001	0.009 ±0.001	<u>0.343</u> ±0.002	0.083 ±0.000	0.227±0.000
FLOWTIME	0.042 ±0.001	0.009 ±0.001	0.284 ±0.002	<u>0.089</u> ±0.000	0.243±0.002

Results Table 2 compares FLOWTIME with the real-world forecasting baselines using test-set CRPS. FLOWTIME achieves the best reported score on Electricity, Exchange, and Solar. On Electricity, its CRPS of 0.042 improves over the second-best TSFlow score of 0.045 by 6.7%. On Exchange, FLOWTIME matches TSFlow at 0.009 and outperforms the remaining statistical, neural, and diffusion-based baselines. The largest margin occurs on Solar, where FLOWTIME obtains 0.284, a 17.2% reduction relative to the next-best TSFlow score of 0.343. On Traffic, FLOWTIME ranks second with 0.089, behind TSFlow at 0.083 and ahead of all remaining baselines; on Wikipedia, its score of 0.243 is competitive but trails WaveNet (0.207) and SSSD (0.209). Overall, the benchmark results show that the autoregressive FM formulation is not limited to synthetic dynamical systems: it also delivers state-of-the-art or competitive probabilistic forecasts across diverse real-world datasets.

5 DISCUSSION

FLOWTIME demonstrates that autoregressive factorization is a natural and effective way to adapt flow matching to probabilistic timeseries forecasting. By replacing a single high-dimensional future-window transport with a sequence of next-step conditional transports, the method preserves the expressive distributional modeling capacity of FM while aligning the generative process with the temporal structure of forecasting. This design makes uncertainty propagation explicit, reduces the dimensionality of each learned transport, and retains efficient teacher-forced training.

The empirical evidence supports this positioning across both controlled and benchmark settings. On stochastic dynamical systems, the autoregressive formulation improves NRMSE over the non-autoregressive baseline in every prediction and extrapolation comparison, while also improving CRPS in most systems. On real-world datasets, FLOWTIME achieves the best CRPS on three of five benchmarks and remains competitive on the remaining two. These results substantiate the claim

that autoregressive flow matching is a practical forecasting framework rather than a construction limited to synthetic data, while the conclusions remain bounded by the evaluated datasets, metrics, and baselines.

The approach also has limitations. Autoregressive sampling is sequential at inference time and therefore introduces computational overhead relative to fully parallel non-autoregressive models, which may constrain use in latency-sensitive forecasting systems. In addition, the present implementation applies flow matching directly in data space, a choice that may be suboptimal for high-dimensional partially observed data. Future work could investigate latent-space formulations, irregularly sampled observations, and additional scientific or decision-oriented forecasting settings. Within the scope of the present evidence, FLOWTIME establishes autoregressive flow matching as a strong direction for improving probabilistic forecast accuracy and uncertainty quantification.

REFERENCES

- Michael S Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv preprint arXiv:2303.08797*, 2023.
- Juan Miguel Lopez Alcaraz and Nils Strodthoff. Diffusion-based time series imputation and forecasting with structured state space models. *arXiv preprint arXiv:2208.09399*, 2022.
- Alexander Alexandrov, Konstantinos Benidis, Michael Bohlke-Schneider, Valentin Flunkert, Jan Gasthaus, Tim Januschowski, Danielle C Maddix, Syama Rangapuram, David Salinas, Jasper Schulz, et al. GluonTS: Probabilistic and neural time series modeling in python. *Journal of Machine Learning Research*, 21(116):1–6, 2020.
- Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.
- Fatoumata Dama and Christine Sinoquet. Time series analysis and modeling to forecast: a survey. In *ArXiv:2104.00164*, 2021.
- Abhyuday Desai, Cynthia Freeman, Zuhui Wang, and Ian Beaver. TimeVAE: A variational auto-encoder for multivariate time series generation. *arXiv preprint arXiv:2111.08095*, 2021.
- Dheeru Dua, Casey Graff, et al. UCI machine learning repository. URL <http://archive.ics.uci.edu/ml>, 7(1):62, 2017.
- Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first International Conference on Machine Learning*, 2024.
- Richard FitzHugh. Impulses and physiological states in theoretical models of nerve membrane. *Biophysical Journal*, 1(6):445–466, 1961. doi: 10.1016/S0006-3495(61)86902-6.
- Jan Gasthaus, Konstantinos Benidis, Yuyang Wang, Syama Sundar Rangapuram, David Salinas, Valentin Flunkert, and Tim Januschowski. Probabilistic forecasting with spline quantile function RNNs. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pp. 1901–1910. PMLR, 2019.
- Will Grathwohl, Ricky TQ Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. FFJORD: Free-Form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations*, 2018.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.
- Yang Hu, Xiao Wang, Lirong Wu, Huatian Zhang, Stan Z. Li, Sheng Wang, and Tianlong Chen. Fm-ts: Flow matching for time series generation. In *ArXiv:2411.07506v1 [cs.LG]*, 2024. URL <http://arxiv.org/abs/2411.07506>.

432 Rob Hyndman, Anne B Koehler, J Keith Ord, and Ralph D Snyder. *Forecasting with Exponential*
433 *Smoothing: The State Space Approach*. Springer Science & Business Media, 2008.

434 Rob J Hyndman and Anne B Koehler. Another look at measures of forecast accuracy. *International*
435 *Journal of Forecasting*, 22(4):679–688, 2006.

436 Maximilian Karl, Maximilian Soelch, Justin Bayer, and Patrick Van der Smagt. Deep varia-
437 tional Bayes filters: Unsupervised learning of state space models from raw data. *arXiv preprint*
438 *arXiv:1605.06432*, 2016.

439 Marcel Kollovich, Abdul Fatir Ansari, Michael Bohlke-Schneider, Jasper Zschiegner, Hao Wang,
440 and Yuyang Bernie Wang. Predict, refine, synthesize: Self-guiding diffusion models for proba-
441 bilistic time series forecasting. *Advances in Neural Information Processing Systems*, 36:28341–
442 28364, 2023.

443 Marcel Kollovich, Marten Lienen, David Lüdke, Leo Schwinn, and Stephan Günnemann. Flow
444 matching with gaussian process priors for probabilistic time series forecasting. In *The Thirteenth*
445 *International Conference on Learning Representations*, 2024.

446 Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. DiffWave: A versatile
447 diffusion model for audio synthesis. *arXiv preprint arXiv:2009.09761*, 2020.

448 Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long- and short-term
449 temporal patterns with deep neural networks. In *The 41st International ACM SIGIR Conference*
450 *on Research & Development in Information Retrieval*, pp. 95–104, 2018.

451 R. Lefever and I. Prigogine. Symmetry breaking instabilities in dissipative systems. II. *The Journal*
452 *of Chemical Physics*, 48(4):1695–1700, 1968. doi: 10.1063/1.1668896.

453 Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: A survey. *Philosophical*
454 *Transactions of the Royal Society A*, 379(2194):20200209, 2021.

455 Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow match-
456 ing for generative modeling. In *The Eleventh International Conference on Learning Representa-*
457 *tions*, 2022.

458 Xingchao Liu, Chengyue Gong, et al. Flow straight and fast: Learning to generate and transfer data
459 with rectified flow. In *The Eleventh International Conference on Learning Representations*, 2022.

460 Edward Lorenz. Deterministic nonperiodic flow. *Journal of Atmospheric Sciences*, 20(2), 1963.

461 Alfred J. Lotka. *Elements of Physical Biology*. Williams & Wilkins, 1925.

462 Caspar Meijer and Lydia Y Chen. The rise of diffusion models in time-series forecasting. *arXiv*
463 *preprint arXiv:2401.03006*, 2024.

464 Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, Ann Lee, Apoorv
465 Vyas, Bowen Shi, Chih-Yao Ma, Ching-Yao Chuang, et al. MovieGen: A cast of media founda-
466 tion models. *arXiv e-prints*, pp. arXiv–2410, 2024.

467 Kashif Rasul, Abdul-Saboor Sheikh, Ingmar Schuster, Urs Bergmann, and Roland Vollgraf. Mul-
468 tivariate probabilistic time series forecasting via conditioned normalizing flows. *arXiv preprint*
469 *arXiv:2002.06103*, 2020.

470 Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. Autoregressive denoising dif-
471 fusion models for multivariate probabilistic time series forecasting. In *International Conference*
472 *on Machine Learning*, pp. 8857–8868. PMLR, 2021.

473 David Salinas, Michael Bohlke-Schneider, Laurent Callot, Roberto Medico, and Jan Gasthaus.
474 High-dimensional multivariate forecasting with low-rank gaussian copula processes. *Advances*
475 *in Neural Information Processing Systems*, 32, 2019.

476 David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic
477 forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3):
478 1181–1191, 2020.

486 Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised
487 learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning*,
488 pp. 2256–2265. PMLR, 2015.

489 Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben
490 Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint*
491 *arXiv:2011.13456*, 2020.

492
493 Ella Tamir, Najwa Laabid, Markus Heinonen, Vikas Garg, and Arno Solin. Conditional flow match-
494 ing for time series modelling. In *ICML Workshop: Structured Probabilistic Inference and Gener-
495 ative Modeling*, 2024.

496 Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. CSDI: Conditional score-based
497 diffusion models for probabilistic time series imputation. *Advances in Neural Information Pro-
498 cessing Systems*, 34:24804–24816, 2021.

499 Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Hugué, Yanlei Zhang, Jarrod Rector-
500 Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models
501 with minibatch optimal transport. *arXiv preprint arXiv:2302.00482*, 2023.

502
503 Aaron Van Den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves,
504 Nal Kalchbrenner, Andrew Senior, Koray Kavukcuoglu, et al. WaveNet: A generative model for
505 raw audio. *arXiv preprint arXiv:1609.03499*, 12, 2016.

506
507 B. van der Pol. On ‘relaxation-oscillations’. *The London, Edinburgh, and Dublin Philosophical*
508 *Magazine and Journal of Science*, 2(11):978–992, 1926. doi: 10.1080/14786442608564127.

509 Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent
510 neural networks. *Neural Computation*, 1(2):270–280, 1989.

511
512 Robert L Winkler, Javier Munoz, José L Cervera, José M Bernardo, Gail Blattenberger, Joseph B
513 Kadane, Dennis V Lindley, Allan H Murphy, Robert M Oliver, and David Ríos-Insua. Scoring
514 rules and the evaluation of probabilities. *Test*, 5:1–60, 1996.

515 Jiexia Ye, Weiqi Zhang, Ke Yi, Yongzi Yu, Ziyue Li, Jia Li, and Fugee Tsung. A survey of time
516 series foundation models: Generalizing time series representation with large language model. In
517 *ArXiv:2104.00164*, 2024.

518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539

A DATASETS

A.1 DYNAMICAL SYSTEMS

We generated training and testing data for five classical stochastic dynamical systems—Lorenz, FitzHugh-Nagumo, Lotka-Volterra, Brusselator, and Van der Pol—by numerically integrating their corresponding stochastic differential equations (SDEs). Each system follows the general form

$$d\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t); \Theta) dt + \sigma d\mathbf{W}(t),$$

where f is the deterministic drift, Θ denotes the system parameters, σ is a constant diffusion term, and $\mathbf{W}(t)$ is a standard Brownian motion. For each system, we sampled 2400 random initial conditions from the uniform distribution specified in Table 3 and generated one trajectory per initial condition by solving the SDE with the Euler-Heun method at a fixed step size. Each trajectory was subsampled at 200 equally spaced time points: the first 75 points form the observation window, the next 75 points form the prediction window, and the final 50 points form the extrapolation window. Of the 2400 generated trajectories, 2000 were used for training and 400 for testing.

Table 3: Specification of the stochastic dynamical systems used for the controlled forecasting experiments. $\mathbf{f}(\mathbf{x})$ denotes the drift function, Θ the system parameters, σ the constant diffusion term, and $p(\mathbf{x}(0))$ the initial condition distribution. Each system was integrated over the interval $[t_0, t_1]$ and sampled at 200 time points using the Euler-Heun method.

System	$\mathbf{f}(\mathbf{x})$	Θ	Σ	$p(\mathbf{x}(0))$	$[t_0, t_1]$
Lorenz	$f_1 = \sigma(x_2 - x_1)$	$\sigma = 10$	(1.5, 1.5, 1.5)	$\mathcal{U}([0, 10])$	[0, 2]
	$f_2 = x_1(\rho - x_3) - x_2$	$\rho = 28$			
	$f_3 = x_1 x_2 - \beta x_3$	$\beta = \frac{8}{3}$			
FitzHugh-Nagumo	$f_1 = x_1 - \frac{x_1^3}{3} - x_2 + I$	$a = 0.7$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 10]
	$f_2 = \frac{x_1 + a - b x_2}{\tau}$	$b = 0.8$			
		$\tau = 12.5$			
Lotka-Volterra	$f_1 = \alpha x_1 - \beta x_1 x_2$	$\alpha = 1.3$	(1.5, 1.5)	$\mathcal{U}([0, 5])$	[0, 20]
	$f_2 = -\delta x_2 + \gamma x_1 x_2$	$\beta = 0.9$			
		$\gamma = 0.8$			
Brusselator	$f_1 = A + x_1^2 x_2 - (B + 1) x_1$	$\alpha = 1.3$	(1.5, 1.5)	$\mathcal{U}([0, 2])$	[0, 20]
	$f_2 = B x_1 - x_1^2 x_2$	$\beta = 0.9$			
		$\gamma = 0.8$			
Van der Pol	$f_1 = x_2$	$\delta = 1.8$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 20]
	$f_2 = \mu(1 - x_1^2) x_2 - x_1$	$A = 1.0$			
		$B = 3.0$			

A.2 REAL-WORLD DATASETS

We used five GluonTS datasets (Alexandrov et al., 2020) to evaluate performance on established real-world forecasting benchmarks. All experiments use the versions preprocessed as in (Salinas et al., 2019); Table 4 summarizes their dimensions, domains, sampling frequencies, training lengths, and prediction horizons.

Table 4: Properties of the real-world forecasting datasets, including dimension n , domain Ω , sampling frequency, total training timesteps, and prediction length.

Dataset	n	Ω	freq.	timesteps	prediction length
Electricity	370	$\mathbb{R}^+$	hourly	5833	24
Exchange	8	$\mathbb{R}^+$	daily	6071	30
Solar	137	$\mathbb{R}^+$	hourly	7009	24
Traffic	963	(0, 1)	hourly	4001	24
Wikipedia	2000	$\mathbb{N}$	daily	792	30

B NON-AUTOREGRESSIVE FLOW MATCHING BASELINE

Problem Setting The non-autoregressive baseline uses the same forecasting setup as Section 3: given past observations $\mathbf{Y}_l$ and covariates $\mathbf{C}$, the objective is to model the conditional distribution $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$. Unlike FLOWTIME, the baseline directly parameterizes the joint distribution of the entire future trajectory $\mathbf{Y}_f \in \mathbb{R}^{f \times n}$ without temporal factorization:

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = p(\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}|\mathbf{Y}_l, \mathbf{C}) \quad (9)$$

This formulation preserves temporal correlations across all future time steps within a single trajectory-level object, but it requires learning a substantially higher-dimensional conditional distribution.

Training We construct a probability path $(p^s(\mathbf{Y}_f))^{0 \leq s \leq 1}$ that transports samples from a prior distribution $p^0(\mathbf{Y}_f) = \mathcal{N}(0, \Sigma)$ to the target distribution $p^1(\mathbf{Y}_f) \approx p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$, where Σ is a block-diagonal covariance matrix. The prior can be interpreted as independent Brownian motion processes across dimensions, with Brownian motion defined as a stochastic process $W: [\tau+1, \tau+f] \rightarrow \mathbb{R}^n$.

The conditional probability path is a linear interpolation bridge with Brownian noise:

$$p^s(\mathbf{Y}_f|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma) \quad (10)$$

where $\mathbf{z} = (\mathbf{Y}_f^0, \mathbf{Y}_f^1)$ with $\mathbf{Y}_f^0 \sim p^0$ and $\mathbf{Y}_f^1 \sim p^1$. The corresponding conditional velocity field is

$$\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0 + \frac{\sigma^2(1-2s)}{2}\Sigma^{-1}(\mathbf{Y}_f - ((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1)) \quad (11)$$

When $\sigma^2 \rightarrow 0$, this expression reduces to the straight-path velocity $\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0$. The baseline learns a neural velocity field ν_θ that operates on the entire future trajectory. Its context encoding remains analogous to Eq. (7):

$$\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f}) \quad (12)$$

where ζ_ϕ processes both past and future covariates. The resulting training objective is

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{Y}_f^s \sim p^s(\mathbf{Y}_f|\mathbf{z})} \left\| \mu(\mathbf{Y}_f^s, s|\mathbf{z}) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s) \right\|^2 \quad (13)$$

where ν_θ is implemented as a sequential neural network (e.g., Transformer or RNN) that processes the entire trajectory. Sampling then requires solving the trajectory-level ODE

$$\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s), \quad \psi(\mathbf{Y}_f, 0) \sim p^0 \quad (14)$$

with numerical solvers. This procedure produces joint samples from $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ without autoregressive decomposition.

Implementation Details This baseline was used in the dynamical-system forecasting experiments. In this setup, ν_θ is a 4-layer bi-directional LSTM with 128 hidden units, and the remaining training parameters match those in Section 4.1. Training to convergence required ~ 2 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Key Differences from Autoregressive Variant The non-autoregressive baseline differs from FLOWTIME in the level at which transport is defined. Whereas FLOWTIME factorizes the joint distribution through a Markovian decomposition across time steps, the non-autoregressive baseline models the full future trajectory $\mathbf{Y}_f$ as a single high-dimensional random variable. This choice preserves cross-temporal dependencies within the trajectory-level representation, but it shifts the learning problem to a more complex distribution over $\mathbb{R}^{n \times f}$. The two approaches also differ computationally and statistically. The non-autoregressive baseline processes entire trajectories through sequential neural networks for the velocity field, enabling parallel generation of all future timepoints at the cost of higher memory requirements. In contrast, FLOWTIME samples future observations sequentially and explicitly propagates uncertainty through the chain of conditional distributions. Thus, the autoregressive variant provides a more interpretable mechanism for uncertainty propagation and efficient window-based conditional modeling under its Markov assumption, whereas the non-autoregressive baseline retains full trajectory-level correlations through a higher-dimensional transport with less explicit error-accumulation dynamics.

Algorithm 3 Non-Autoregressive Flow Matching: Training

Require: Dataset $\mathcal{D}$, networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
- 2: Sample $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$ and $(\mathbf{Y}_f^1, \mathbf{Y}_l, \mathbf{C}) \sim \mathcal{D}$
- 3: Set $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f})$
- 4: Sample $s \sim \mathcal{U}(0, 1)$
- 5: $\mathbf{Y}_f^s \sim \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma)$
- 6: Compute target $\mu(\mathbf{Y}_f^s, s|\mathbf{z})$
- 7: Update θ, ϕ to minimize $\|\mu(\cdot) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|_{\Sigma^{-1}}^2$
- 8: **end while**

Algorithm 4 Non-Autoregressive Flow Matching: Sampling

Require: $\mathbf{Y}_l, \mathbf{C}$, networks ν_θ, ζ_ϕ

- 1: Encode context: $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C})$
- 2: Sample initial trajectory $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$
- 3: Solve $\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)$
- 4: Return $\psi(\mathbf{Y}_f, 1)$

C EVALUATION METRICS

This section defines the two metrics used to evaluate forecast quality. Given an n -dimensional future time series $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$ and either its predictive distribution or samples from the probabilistic model, we report the Continuous Ranked Probability Score (CRPS) and the Normalized Root Mean Squared Error (NRMSE).

C.1 CONTINUOUS RANKED PROBABILITY SCORE (CRPS)

The CRPS is a proper scoring rule for evaluating probabilistic forecasts. For a univariate random variable X with cumulative distribution function (CDF) F_X and an observed value x , the CRPS is defined as

$$\text{CRPS}(F_X, x) = \int_{-\infty}^{+\infty} (F_X(y) - \mathbf{1}\{y \geq x\})^2 dy, \quad (15)$$

where $\mathbf{1}\{\cdot\}$ is the indicator function. The CRPS evaluates the alignment between the full predictive distribution and the realized observation x ; lower values indicate predictive distributions that are better calibrated and more accurate.

For time series forecasting, we compute CRPS at each horizon and average over the forecast window:

$$\text{CRPS}_{\text{avg}} = \frac{1}{f} \sum_{k=1}^f \text{CRPS}(F_{X_{\tau+k}}, y_{\tau+k}), \quad (16)$$

where $F_{X_{\tau+k}}$ is the predicted CDF, or the empirical CDF induced by forecast samples, at horizon k , and $y_{\tau+k}$ is the corresponding ground-truth observation.

C.2 NORMALIZED ROOT MEAN SQUARED ERROR (NRMSE)

The Root Mean Squared Error (RMSE) measures point-forecast accuracy. For predictions $\{\hat{y}_{\tau+1}, \dots, \hat{y}_{\tau+f}\}$ and corresponding ground-truth values $\{y_{\tau+1}, \dots, y_{\tau+f}\}$, RMSE is defined as

$$\text{RMSE} = \sqrt{\frac{1}{f} \sum_{k=1}^f (\hat{y}_{\tau+k} - y_{\tau+k})^2}. \quad (17)$$

To make RMSE scale-invariant and comparable across datasets or time series with different magnitudes, we report Normalized RMSE (NRMSE). One common normalization divides by the standard deviation of the ground-truth series, σ_y :

$$\text{NRMSE} = \frac{\text{RMSE}}{\sigma_y}, \quad (18)$$

where σ_y is the sample standard deviation of the observations. Alternative normalizations, such as the data range ($\max(y) - \min(y)$), may be appropriate in other applications. Under the reported normalization, lower NRMSE indicates more accurate predictions relative to the variability of the time series.